

TEZPUR UNIVERSITY

Networking Internship Program

Assignment 6

GUI-Based Multi-Client Chat Application Using TCP

Submitted by

Bhargav Jyoti Saikia

Roll Number: CSB24022

Department of Computer Science and Engineering
Tezpur University

Submission Date: July 9, 2026

Table of Contents

1. Objective	3
2. Introduction.....	3
3. Software and System Requirements	4
4. Network Topology	5
5. GUI Design	6
6. System Architecture	9
7. Components Reused from Assignment 5	10
8. GUI Design Decisions	11
9. Background Message Handling.....	12
10. Testing Results	13
11. Wireshark Verification	17
12. Reflection Questions.....	20
13. Conclusion	21
14. References.....	21

1. Objective

The objective of Assignment 6 is to convert the terminal-based TCP chat application developed in Assignment 5 into a graphical desktop application using Python's Tkinter library, while reusing the existing TCP server implementation without modification to its core networking logic. This assignment introduces GUI programming concepts, event-driven design, multithreading for responsive user interfaces, and the practical challenges of integrating a graphical front end with an already functioning socket-based back end.

The specific goals of this assignment are to build a login window for username and server IP entry, construct a graphical chat window with a scrollable message area, implement an online user list that updates dynamically, support both broadcast and private messaging through the GUI, handle incoming messages on a background thread so that the interface never freezes, and verify the correctness of the implementation using Wireshark packet captures on port 5000.

2. Introduction

2.1 TCP Communication

Transmission Control Protocol (TCP) is a connection-oriented, reliable transport-layer protocol that guarantees ordered and error-checked delivery of a stream of bytes between two hosts. Before any data is exchanged, TCP performs a three-way handshake (SYN, SYN-ACK, ACK) to establish a connection, and it uses sequence numbers, acknowledgements, and retransmission timers to ensure that no data is lost or duplicated in transit. Because TCP presents the application with a reliable, ordered byte stream, it is well suited to a chat application, where messages must arrive completely and in the order they were sent.

In this assignment, the chat server listens on TCP port 5000 and accepts connections from multiple clients simultaneously. Each client maintains a single persistent TCP connection to the server for the duration of the chat session, and all communication, including login, broadcast messages, private messages, and the online user list, is carried over this one connection using simple text-based framing.

2.2 Client-Server Architecture

The application follows a classic client-server architecture in which a single central server coordinates communication between multiple independent clients. The server accepts incoming TCP connections, assigns each client a dedicated handler thread, and maintains a

shared dictionary of connected users and their sockets. When a client sends a broadcast message, the server forwards it to every other connected client; when a client sends a private message, the server routes it only to the specified recipient. This centralized model avoids the complexity of direct peer-to-peer connections between clients and allows the server to maintain authoritative state, such as the current list of online users and the chat history log.

2.3 Why a GUI Improves Usability Over a Terminal-Based Application

The terminal-based client developed in Assignment 5 required users to type raw commands and read plain text output scrolling in a console window, which is functional but not user-friendly. A graphical interface improves usability in several concrete ways. First, it separates incoming chat messages, the online user list, and the message input box into distinct visual regions, so the user does not have to mentally parse mixed output. Second, buttons for common actions such as Connect, Send, and Disconnect remove the need to remember command syntax. Third, a dedicated Listbox widget for online users gives an immediate, always-visible view of who is currently connected, which a terminal cannot provide without repeated manual queries. Finally, a status indicator and message boxes give immediate visual feedback on connection state and errors, making the application accessible to users unfamiliar with command-line tools.

3. Software and System Requirements

The following software components were used to design, implement, and test the GUI-based chat application:

Component	Purpose
Ubuntu Linux (VirtualBox)	Host operating system for running the Mininet emulated network
Python 3	Programming language used for both server and client implementation
Tkinter	Standard Python GUI toolkit used to build the client interface
Socket Programming (socket module)	Provides the TCP client and server communication primitives
Mininet	Emulates a network topology with one server host and four client hosts
Wireshark	Captures and analyzes TCP packets exchanged on port 5000

Visual Studio Code	Code editor used for writing and debugging server.py and client_gui.py
--------------------	--

4. Network Topology

The application was tested using a Mininet-emulated network created with the command shown below, which produces a single-switch star topology connecting one server host and four client hosts.

```
sudo mn --topo single,5
```

The resulting topology assigns the following roles:

- h1 — Chat Server (runs server.py, listening on 0.0.0.0:5000)
- h2 — Client A (runs client_gui.py, connects as user Bhargav)
- h3 — Client B (runs client_gui.py, connects as user Alice)
- h4 — Client C (runs client_gui.py, connects as user John)
- h5 — Client D (runs client_gui.py, connects as user Charlie)

All five hosts are connected through a single OVS switch (s1), which allows every client to reach the server at IP address 10.0.0.1 on port 5000. Connectivity between hosts was verified using the nodes, net, and pingall commands within the Mininet CLI before starting the server and client applications.

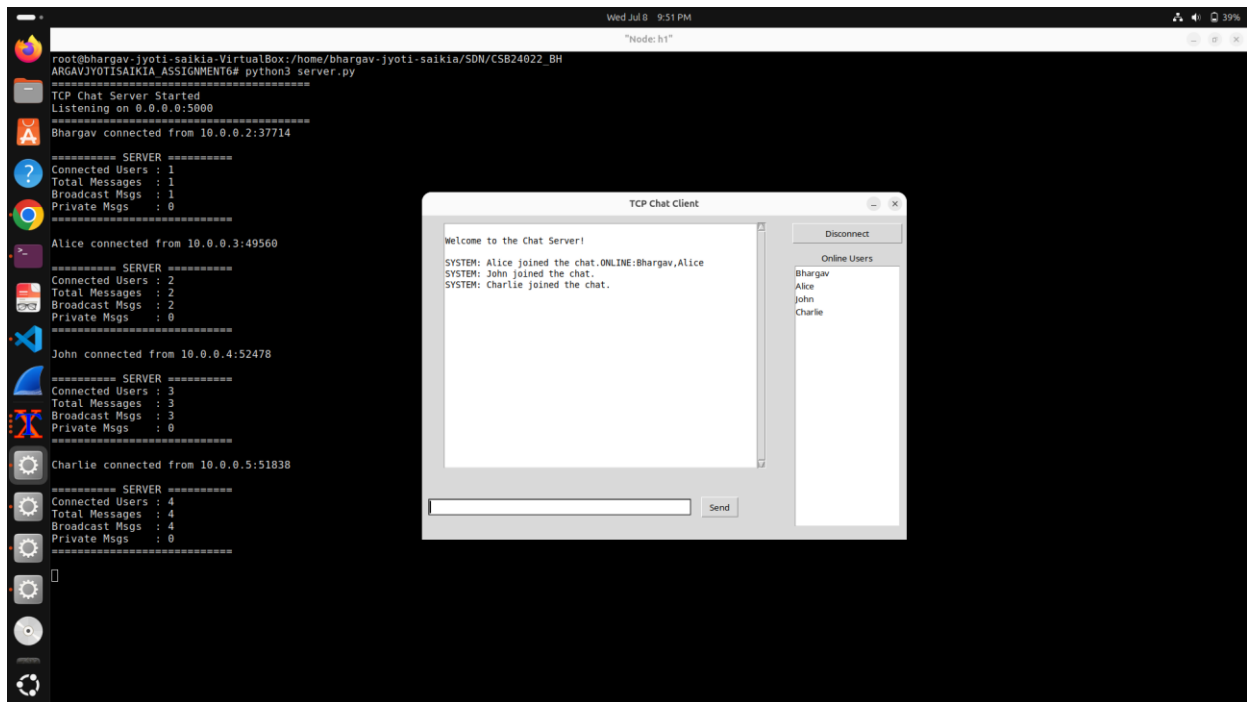


Figure 4.1: Mininet topology (single,5) with the chat server running on h1 and a connected GUI client, terminal log showing users joining.

5. GUI Design

The client_gui.py application is structured around two main Tkinter frames that are shown and hidden depending on connection state: a login_frame, shown at startup, and a chat_frame, shown after a successful connection. Each GUI component is described below with reference to the corresponding code in client_gui.py.

5.1 Login Window

The login window is the first screen presented to the user. It is built as a tk.Frame named login_frame and contains a title label, a Server IP entry, a Username entry, a status label, and a Connect button.

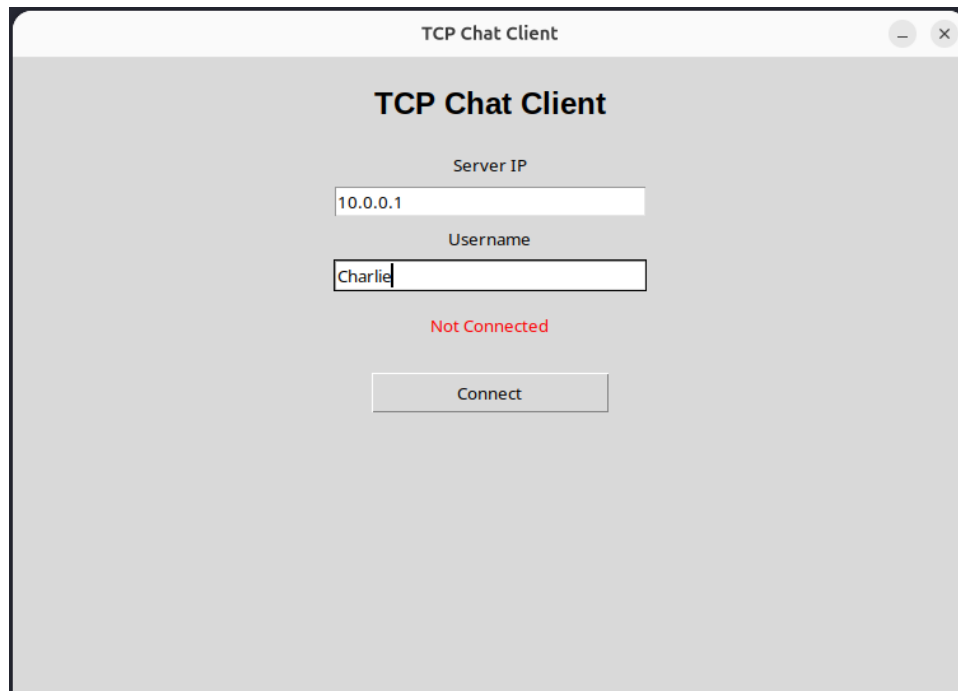


Figure 5.1: Login window with Server IP field pre-filled and Username field awaiting input, and a red “Not Connected” status label.

5.2 Username Entry

The `username_entry` widget is a standard `tk.Entry` that accepts the display name the user wishes to chat under. On clicking `Connect`, the `connect_server()` function reads this value with `username_entry.get().strip()` and validates that it is not empty before proceeding. The username is transmitted to the server immediately after the initial handshake, via `client.send(username.encode())`.

5.3 Server IP Entry

The `host_entry` widget allows the user to specify the IP address of the chat server. In the code it is pre-populated with a default value of "10.0.0.1" using `host_entry.insert(0, "10.0.0.1")`, which corresponds to host `h1` in the Mininet topology. This makes it convenient to connect to the server without retyping the address for every test run, while still allowing the field to be edited for other topologies.

5.4 Connect Button

The `connect_btn` button is bound to the `connect_server()` function via `command=connect_server`. This function performs input validation, opens a TCP socket with `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`, connects to `(host, PORT)`, completes the username handshake, and then switches the visible frame from `login_frame` to

chat_frame using pack_forget() and pack() respectively. It also starts the background receiving thread at this point.

5.5 Chat Window

The chat_frame contains two child frames: left_frame, which holds the scrollable chat area, the message entry box, and the Send button, and right_frame, which holds the Disconnect button and the online users list. This left/right split keeps the conversation area visually separate from session controls.

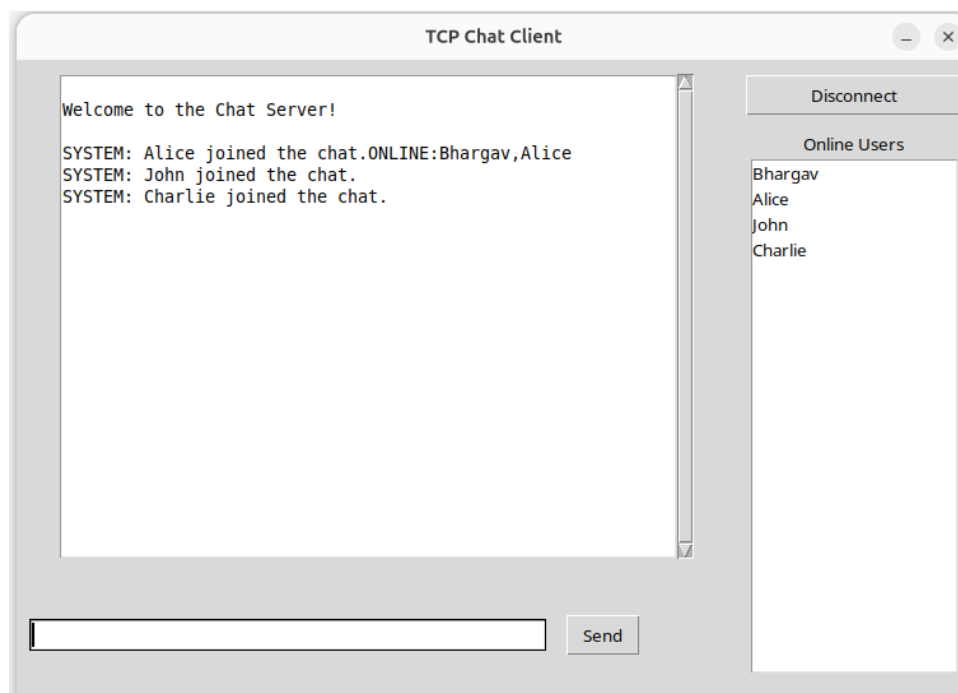


Figure 5.2: Full chat window immediately after connecting, showing the join notifications for Alice, John, and Charlie and the populated Online Users list.

5.6 Scrollable Chat Area

The chat_box widget is a scrolledtext.ScrolledText instance configured with state="disabled" so that the user cannot directly type into the display area. Whenever a new message is received by the background receive() thread, the widget is temporarily set to state="normal", the message is appended with chat_box.insert(tk.END, message + "\n"), the view is scrolled to the bottom with chat_box.see(tk.END), and the widget is returned to state="disabled". This pattern keeps the chat log read-only from the user's perspective while still allowing programmatic updates.

5.7 Message Input Box

The `message_entry` widget is a `tk.Entry` where the user types outgoing messages. It is bound to the Return (Enter) key via `message_entry.bind("<Return>", ...)`, so that pressing Enter has the same effect as clicking Send, which improves the typing workflow for a chat application.

5.8 Send Button

The `send_btn` button is bound to `send_message()`, which reads and strips the text from `message_entry`, checks that the client socket exists and the message is non-empty, transmits the message with `client.send(message.encode())`, and then clears the input box with `message_entry.delete(0, tk.END)`. Error handling around the send call displays a `messagebox.showerror` dialog if transmission fails.

5.9 Disconnect Button

The `disconnect_btn` button is bound to `disconnect()`, which sends a final `"/quit"` command to the server, closes the socket, and then destroys the root Tkinter window, cleanly ending the client session. The same function is also invoked automatically when the window is closed via the operating system's close button, through the `WM_DELETE_WINDOW` protocol handler `on_close()`.

5.10 Online User List

The `online_list` widget is a `tk.Listbox` on the right side of the chat window. Whenever the `receive()` thread detects a message beginning with the prefix `"ONLINE:"`, it parses the comma-separated usernames that follow, clears the Listbox with `online_list.delete(0, tk.END)`, and repopulates it with the current set of connected users. This gives every client a live, automatically refreshed view of who is online.

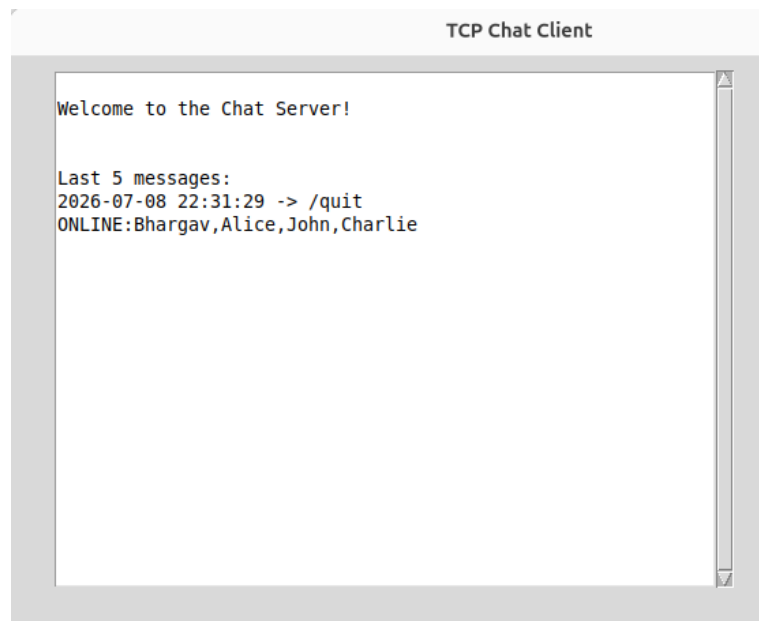


Figure 5.3: Online Users panel updated to show four connected users (Bhargav, Alice, John, Charlie).

5.11 Status Indicator

The `status_label` widget displays the text "Not Connected" in red before a connection is established. Upon a successful connection inside `connect_server()`, the label is updated to "Connected" in green using `status_label.config(text="Connected", fg="green")`, giving the user immediate visual confirmation that the TCP handshake and login sequence succeeded.

6. System Architecture

The overall system architecture follows a layered flow: multiple GUI clients send requests to a single TCP server, the server routes each message either as a broadcast or as a private message, and the resulting messages are pushed back down to the appropriate client GUIs for display. This flow is illustrated in Figure 6.1.

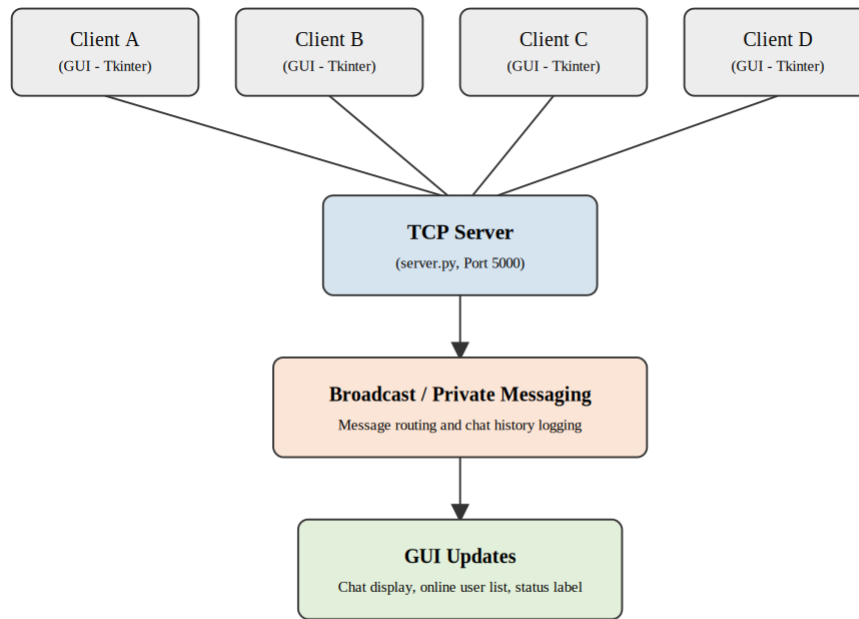


Figure 6.1: System architecture showing multiple clients connecting to a single TCP server, which performs broadcast/private message routing before pushing updates back to client GUIs.

Each client GUI runs two threads: the main thread, which drives the Tkinter event loop and handles user-initiated actions such as clicking Send or Disconnect, and a background receive() thread, which blocks on client.recv() and pushes incoming data into the GUI widgets. The server, in turn, runs one dedicated handle_client() thread per connected client, all of which share the clients dictionary protected by a threading.Lock to guard against race conditions when multiple clients connect, disconnect, or send messages concurrently.

7. Components Reused from Assignment 5

In line with the assignment's requirement to extend rather than rebuild the Assignment 5 implementation, the following components were reused with no or minimal modification:

- TCP socket programming: the underlying socket.socket(AF_INET, SOCK_STREAM) client-server connection logic is unchanged from Assignment 5.
- Server implementation: server.py retains the same handle_client() per-connection thread model, the same clients and client_info dictionaries, and the same threading.Lock synchronization strategy.

- Broadcast messaging: the `broadcast()` function, which iterates over all connected clients except the sender and forwards the message, was reused unchanged.
- Private messaging: the `private_message()` function, which looks up the receiver in the clients dictionary and sends a "[PRIVATE]" tagged message, was reused unchanged, and is invoked from the client GUI using the same `"/msg <username> <message>"` command syntax.
- Chat history: the `save_history()` function and the `chat_history.csv` logging format, which records timestamp, sender, receiver, message type, and message text for every broadcast and private message, were reused unchanged.
- Online user management: the `send_online_users()` and `update_online_users()` functions, which build and broadcast an "ONLINE:" prefixed comma-separated list of connected usernames, were reused unchanged; only the client-side parsing of this message is new, feeding the GUI Listbox instead of printing to a terminal.
- Threading: the one-thread-per-client model on the server, and the concept of a dedicated background thread for receiving data, were both carried over; on the client side this concept was extended by introducing a background `receive()` thread specifically to keep the Tkinter GUI responsive.

The only substantial new work in this assignment was the `client_gui.py` front end itself, which replaces the terminal input/output loop of Assignment 5 with Tkinter widgets while calling into exactly the same wire protocol (plain-text messages, the `"/msg"` and `"/quit"` commands, and the "ONLINE:" status broadcast).

8. GUI Design Decisions

Why Tkinter was selected: Tkinter is included in the standard Python distribution, so it requires no additional installation and keeps the assignment's dependency footprint minimal. It also provides all of the widgets required by the assignment brief, such as Entry, Button, Listbox, and scrolledtext, and integrates cleanly with the existing socket-based networking code without requiring an asynchronous or callback-based rewrite of the client logic.

Why ScrolledText was used: The ScrolledText widget from `tkinter.scrolledtext` combines a Text widget with an attached vertical Scrollbar in a single component. This was the natural choice for the chat display because conversation history can grow arbitrarily long, and users

need to be able to scroll back through earlier messages while new messages continue to arrive at the bottom of the log.

Why Listbox was used: A Listbox provides a simple, always-visible, line-per-item display that maps directly onto the server's comma-separated "ONLINE:" user list. It can be fully cleared and repopulated in two lines of code (`delete(0, tk.END)` followed by a loop of `insert(tk.END, user)`), which matches the way the server pushes a complete, refreshed user list rather than incremental join/leave deltas.

Why Entry widgets were used: Entry widgets were used for the Server IP, Username, and Message input fields because each of these is a single line of free-form text that the user types and submits, which is exactly the use case Entry is designed for. Entry also supports key bindings, which was used to bind the Return key to message sending.

Why Buttons were used: Buttons were used for Connect, Send, and Disconnect because each represents a discrete, user-initiated action rather than continuous input. Binding each button's command parameter directly to a Python function (`connect_server`, `send_message`, `disconnect`) keeps the GUI event handling explicit and easy to trace back to the underlying networking calls.

Why threading was necessary: Tkinter runs a single-threaded event loop (`mainloop()`) that must remain free to process user input and redraw the interface. The socket call `client.recv()` is blocking: it does not return until data arrives from the server. If `recv()` were called directly inside the main thread, the entire GUI would freeze between messages. A separate background thread was therefore required to perform the blocking receive operation without stalling the Tkinter event loop.

9. Background Message Handling

9.1 Separate Receiving Thread

After a successful connection, `connect_server()` starts a daemon thread targeting the `receive()` function: `threading.Thread(target=receive, daemon=True).start()`. This thread runs for the entire lifetime of the connection and is solely responsible for calling `client.recv(4096)` and interpreting incoming data, completely independent of the Tkinter main thread.

9.2 Non-Blocking GUI

Because all blocking socket operations after the initial connection happen on the background thread, the Tkinter main thread is free to respond immediately to button clicks, key presses, and window redraw events. The user can continue typing a new message, scroll through history, or click Disconnect at any moment, even while the receive thread is waiting on incoming data.

9.3 Event-Driven Communication

Rather than polling for new messages on a timer, the client relies on the blocking nature of `recv()` to wake up precisely when the server sends data. Each time a message is received, the code inspects its content, distinguishes an "ONLINE:" status update from an ordinary chat line, and immediately updates the corresponding widget, effectively treating each incoming packet as an event that triggers a specific GUI update.

9.4 Continuous Message Reception

The `receive()` function runs an infinite `while True` loop that keeps calling `client.recv()` until the connection is closed or an exception occurs, at which point the loop breaks and the socket is closed. This ensures that the client can continuously receive an unbounded stream of broadcast messages, private messages, join/leave notifications, and online-user updates for as long as the session remains active.

10. Testing Results

The application was tested inside the Mininet single,5 topology using four simultaneous GUI clients (Bhargav, Alice, John, and Charlie) connecting to the server running on host h1. The results of all functional test cases are summarized in Table 10.1.

Test Case	Description	Result
Successful Login	Client connects with valid Server IP and Username; status label turns green ("Connected")	PASS
Broadcast Messaging	Message typed by one client is received and displayed by all other connected clients	PASS
Private Messaging	Message sent via <code>/msg <username> <text></code> is delivered only to the specified recipient and echoed to the sender as "[You -> user]"	PASS
Online User Update	Online Users Listbox refreshes automatically when a client joins or leaves	PASS

User Joining	New client connection triggers a SYSTEM join broadcast visible to all existing clients	PASS
User Leaving	Client disconnect (/quit) triggers a SYSTEM leave broadcast and removes the user from the online list	PASS
Disconnect Functionality	Disconnect button closes the socket cleanly and closes the client window	PASS
Background Message Handling	GUI remains responsive (typing, scrolling) while messages are received on a separate thread	PASS

Table 10.1: Summary of functional test cases and results.

10.1 Multiple Clients Connected

Figure 10.1 shows the Mininet terminal with the server statistics block alongside a connected GUI client displaying broadcast messages exchanged between Bhargav, Alice, and John, with all users visible in the Online Users list.

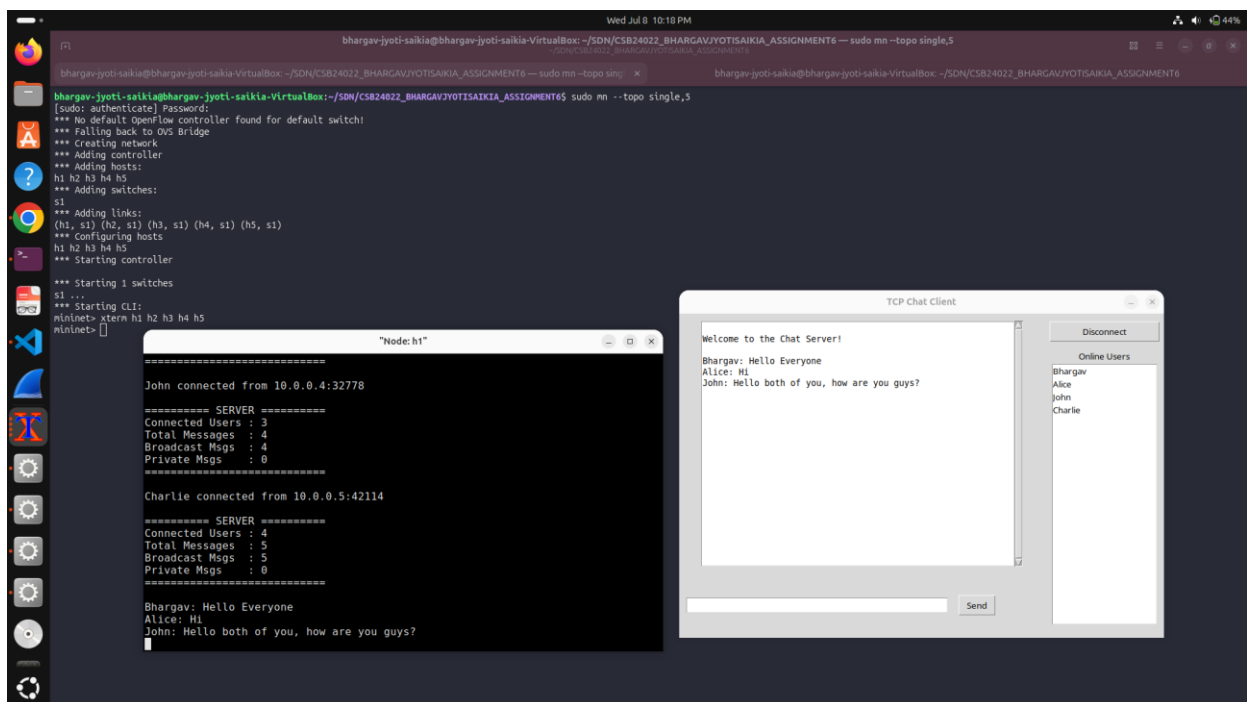


Figure 10.1: Server-side terminal statistics and client GUI showing a broadcast conversation among Bhargav, Alice, and John.

10.2 Broadcast Messaging

Figure 10.2 shows three simultaneously open client windows exchanging broadcast greetings, confirming that a message typed by one client ("Bhargav: Hello Everyone") is delivered to and displayed identically on every other connected client's chat window.

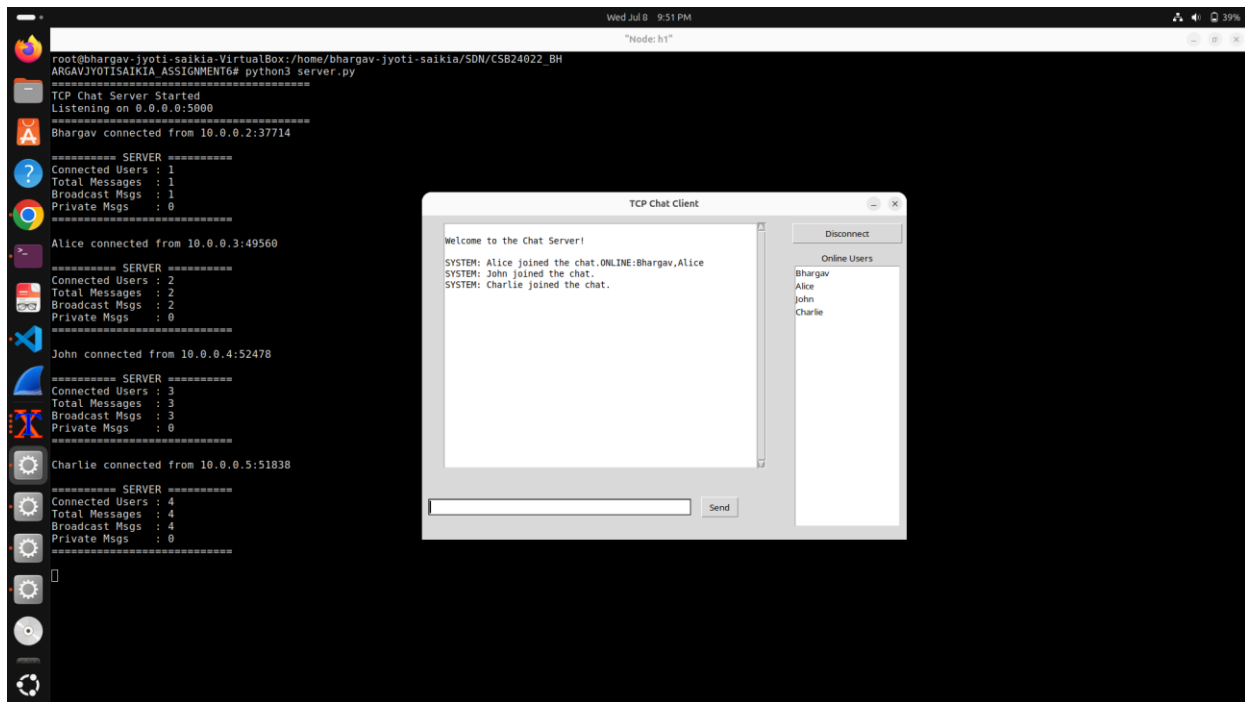


Figure 10.2 (reference): Multiple GUI clients receiving the same broadcast messages, confirming correct message fan-out from the server.

10.3 Private Messaging

Private messaging was tested using the `/msg <username> <message>` command from the message input box. Figure 10.3 shows three client windows: John's window displays the outgoing private exchange with Bhargav ("`[You -> John]`" and "`[PRIVATE] John`"), while a separate window shows Alice's private exchange with Bhargav, and Bhargav's own window shows both private conversations without either message leaking to Charlie, confirming that private messages are routed only to the intended recipient.

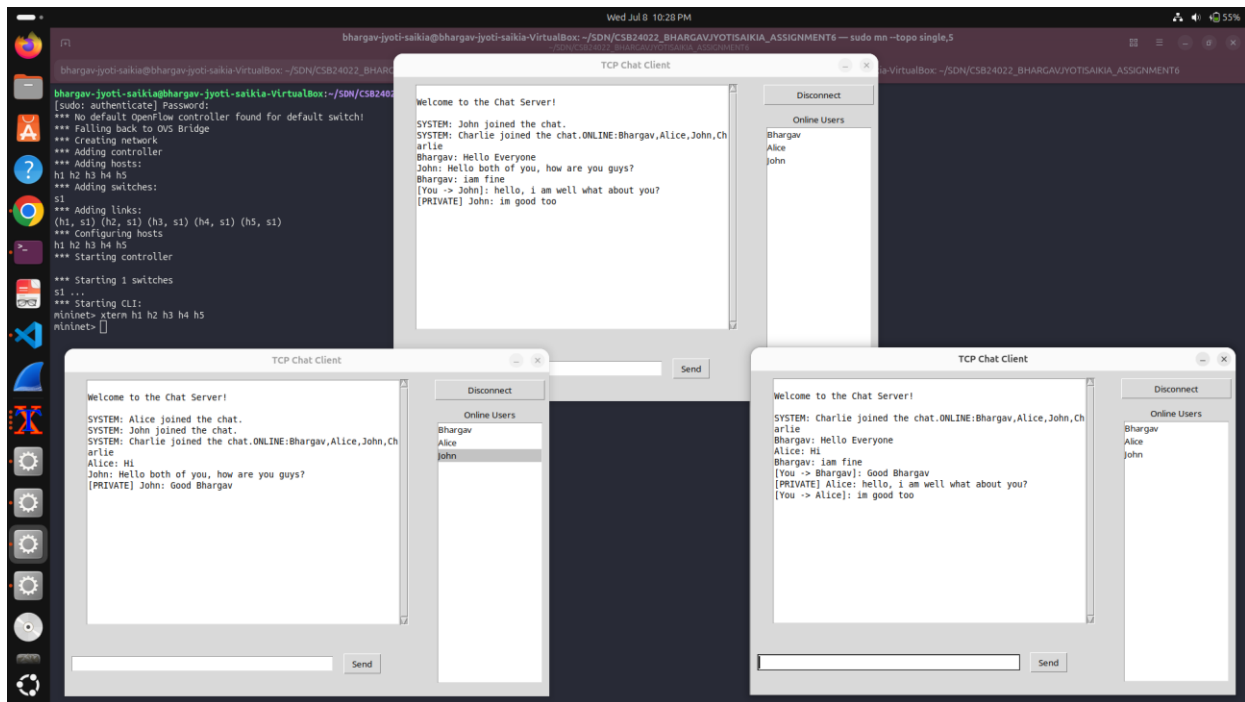


Figure 10.3: Three concurrent client windows demonstrating private messaging between Bhargav, John, and Alice, with correct one-to-one delivery.

10.4 User Joining

Figure 10.4 shows the chat log immediately after Charlie's client connects: a "SYSTEM: Charlie joined the chat." broadcast appears in every other client's window, and the Online Users list is updated to include Charlie alongside Bhargav, Alice, and John.

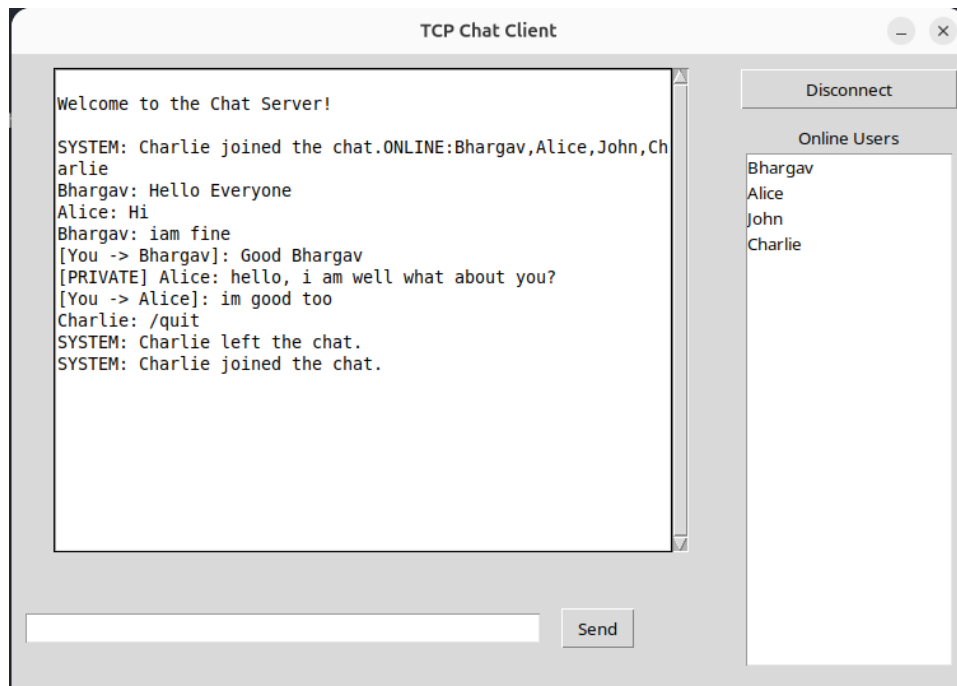


Figure 10.4: SYSTEM join notification and updated Online Users list after Charlie connects.

10.5 User Leaving

Figure 10.5 shows the chat log after Charlie sends `/quit` and disconnects: a "SYSTEM: Charlie left the chat." broadcast is shown, and Charlie is removed from the Online Users list, leaving only Bhargav, Alice, and John.

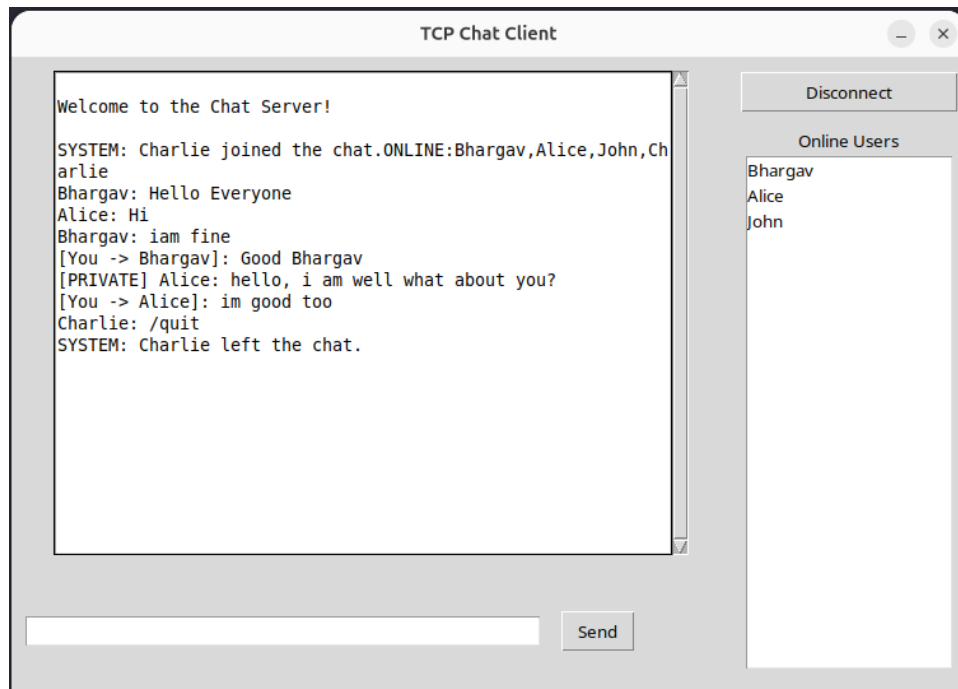


Figure 10.5: SYSTEM leave notification and updated Online Users list after Charlie disconnects.

11. Wireshark Verification

Network traffic was captured on the switch interface `s1-eth1` using the display filter `tcp.port == 5000`, which isolates all TCP traffic exchanged between the chat clients and the server. The following subsections present and explain the captured evidence for connection establishment, broadcast messaging, private messaging, and client disconnection.

11.1 TCP Connection Establishment

Figure 11.1 shows the Wireshark capture window with the filter `tcp.port == 5000` applied. The first two frames show client `10.0.0.2` sending a PSH, ACK segment to the server at `10.0.0.1` on port `5000`, followed by the server's ACK, representing the username handshake immediately after the TCP connection was opened. Subsequent frames show the server sending PSH, ACK segments (containing the welcome message) to clients `10.0.0.3`, `10.0.0.4`, and `10.0.0.5` in turn, each acknowledged by the respective client.

The image shows a Wireshark packet capture window titled 'Capturing from st-eth1'. The filter is 'tcp.port == 5000'. The packet list shows several TCP connections. The packet details pane for packet 1 shows a PSH, ACK segment from 10.0.0.2 to 10.0.0.1, Seq=1, Ack=1, Win=83, Len=5, TSval=738839533, TSecr=738839533. The packet bytes pane shows the raw data of the segment.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	TCP	54	35860 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=83 Len=5 TSval=738839533 TSecr=738839533
2	0.000000000	10.0.0.1	10.0.0.2	TCP	66	5000 → 35860 [ACK] Seq=1 Ack=6 Win=85 Len=0 TSval=285465671 TSecr=738839533
3	0.000377743	10.0.0.1	10.0.0.3	TCP	88	5000 → 54044 [PSH, ACK] Seq=1 Ack=1 Win=85 Len=14 TSval=4292435323 TSecr=777282833
4	0.000392889	10.0.0.1	10.0.0.4	TCP	88	5000 → 32778 [PSH, ACK] Seq=1 Ack=1 Win=85 Len=14 TSval=2707586486 TSecr=2934256689
5	0.000401892	10.0.0.1	10.0.0.5	TCP	88	5000 → 59234 [PSH, ACK] Seq=1 Ack=1 Win=85 Len=14 TSval=1275836330 TSecr=210211847
6	0.002507180	10.0.0.3	10.0.0.1	TCP	66	54044 → 5000 [ACK] Seq=1 Ack=15 Win=83 Len=0 TSval=778319815 TSecr=4292435323
7	0.002525250	10.0.0.4	10.0.0.1	TCP	66	32778 → 5000 [ACK] Seq=1 Ack=15 Win=83 Len=0 TSval=2935292872 TSecr=2707586486
8	0.002566353	10.0.0.5	10.0.0.1	TCP	66	59234 → 5000 [ACK] Seq=1 Ack=15 Win=83 Len=0 TSval=2103147991 TSecr=1275836330

Figure 11.1: Wireshark capture of the connection handshake and initial welcome-message exchange for four clients on TCP port 5000.

A closer view of the three-way handshake for the fourth client (10.0.0.5) is shown in Figure 11.2, confirming the SYN, SYN-ACK, and ACK sequence that establishes the TCP connection before any application data is exchanged.

The image shows a detailed view of the three-way handshake for client 10.0.0.5. The packet list shows three packets: a SYN segment from 10.0.0.5 to 10.0.0.1, a SYN-ACK segment from 10.0.0.1 to 10.0.0.5, and an ACK segment from 10.0.0.5 to 10.0.0.1.

No.	Time	Source	Destination	Protocol	Length	Info
25	130.562113534	10.0.0.3	10.0.0.1	TCP	94	54044 → 5000 [PSH, ACK] Seq=1 Ack=15 Win=83 Len=28 TSval=778449575 TSecr=4292435323
26	130.562257933	10.0.0.1	10.0.0.2	TCP	98	5000 → 35860 [PSH, ACK] Seq=1 Ack=6 Win=85 Len=32 TSval=285596233 TSecr=738839533
27	130.562274189	10.0.0.1	10.0.0.3	TCP	98	5000 → 54044 [PSH, ACK] Seq=15 Ack=29 Win=85 Len=33 TSval=4292565885 TSecr=778449575
28	130.564736552	10.0.0.3	10.0.0.1	TCP	66	54044 → 5000 [ACK] Seq=29 Ack=48 Win=83 Len=0 TSval=778449578 TSecr=4292565885
29	130.565445516	10.0.0.2	10.0.0.1	TCP	66	35860 → 5000 [ACK] Seq=6 Ack=33 Win=83 Len=0 TSval=138078097 TSecr=285596233

Figure 11.2: Detailed view of the SYN / SYN-ACK / ACK three-way handshake for client 10.0.0.5 connecting to the server on port 5000.

11.2 Broadcast Message Capture

Figure 11.3 shows the server (10.0.0.1) sending PSH, ACK segments to clients 10.0.0.2, 10.0.0.3, and 10.0.0.4 with matching sequence numbers and lengths, which corresponds to the server broadcasting a single chat message to every connected client immediately after receiving it from the sender.

The image shows a broadcast message fan-out from the server (10.0.0.1) to multiple clients. The packet list shows several PSH, ACK segments from 10.0.0.1 to various clients, all with the same sequence number (181) and acknowledgment number (6).

No.	Time	Source	Destination	Protocol	Length	Info
124	384.541788798	10.0.0.5	10.0.0.1	TCP	66	43504 → 5000 [FIN, ACK] Seq=13 Ack=155 Win=42496 Len=0 TSval=806498257 TSecr=2648614837
125	384.542781385	10.0.0.1	10.0.0.2	TCP	96	5000 → 35860 [PSH, ACK] Seq=181 Ack=6 Win=85 Len=30 TSval=285856214 TSecr=739224034
126	384.542822339	10.0.0.1	10.0.0.3	TCP	96	5000 → 54044 [PSH, ACK] Seq=181 Ack=6 Win=85 Len=30 TSval=285856214 TSecr=739224034
127	384.550143855	10.0.0.1	10.0.0.4	TCP	96	5000 → 54044 [PSH, ACK] Seq=181 Ack=6 Win=85 Len=30 TSval=285856214 TSecr=739224034
128	384.550173974	10.0.0.1	10.0.0.5	TCP	96	5000 → 59000 [PSH, ACK] Seq=181 Ack=6 Win=85 Len=30 TSval=285856214 TSecr=739224034
129	384.550189866	10.0.0.1	10.0.0.4	TCP	96	5000 → 54044 [PSH, ACK] Seq=181 Ack=6 Win=85 Len=30 TSval=285856214 TSecr=739224034
130	384.550542769	10.0.0.1	10.0.0.1	TCP	66	32778 → 5000 [ACK] Seq=1 Ack=193 Win=83 Len=0 TSval=2935677429 TSecr=2707586486
131	384.550585615	10.0.0.1	10.0.0.2	TCP	91	5000 → 35860 [PSH, ACK] Seq=211 Ack=6 Win=85 Len=25 TSval=285856222 TSecr=739224077
132	384.550594551	10.0.0.1	10.0.0.1	TCP	66	35860 → 5000 [ACK] Seq=6 Ack=236 Win=83 Len=0 TSval=739224085 TSecr=285856222
133	384.550599577	10.0.0.1	10.0.0.3	TCP	91	5000 → 54044 [PSH, ACK] Seq=226 Ack=29 Win=85 Len=25 TSval=4292619874 TSecr=778703564
134	384.550605745	10.0.0.1	10.0.0.1	TCP	66	54044 → 5000 [ACK] Seq=29 Ack=251 Win=83 Len=0 TSval=778703565 TSecr=4292619874
135	384.550609919	10.0.0.1	10.0.0.4	TCP	91	5000 → 32778 [PSH, ACK] Seq=193 Ack=1 Win=85 Len=25 TSval=2707586486 TSecr=2935677429
136	384.550612548	10.0.0.1	10.0.0.1	TCP	66	32778 → 5000 [ACK] Seq=1 Ack=210 Win=83 Len=0 TSval=2935677421 TSecr=2707586486
137	384.550746482	10.0.0.1	10.0.0.5	TCP	66	5000 → 43504 [FIN, ACK] Seq=135 Ack=14 Win=43526 Len=0 TSval=806498266 TSecr=806498257
138	384.550770721	10.0.0.5	10.0.0.1	TCP	66	43504 → 5000 [ACK] Seq=14 Ack=156 Win=42496 Len=0 TSval=806498266 TSecr=2648642868

Figure 11.3: Broadcast message fan-out, showing the server pushing the same PSH, ACK segment to multiple connected clients.

11.3 Private Message Capture

Figure 11.4 shows a PSH, ACK segment (frame 25) sent from client 10.0.0.3 to the server carrying a private message command, followed by the server relaying PSH, ACK segments to 10.0.0.1 and 10.0.0.2 (frames 26 and 27), and the corresponding ACK confirmations. Because these segments are addressed only to the sender and the intended recipient, and not to every connected client, this capture confirms that private messages are routed point-to-point rather than broadcast.

77	256.476571486	10.0.0.5	10.0.0.1	TCP	74	43504	→	5000	[SYN, ACK] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=806370191 TSecr=0 WS=512
78	256.476619017	10.0.0.1	10.0.0.5	TCP	74	5000	→	43504	[SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=2648514794 TSecr=806370191 WS=512
79	256.477175606	10.0.0.5	10.0.0.1	TCP	66	43504	→	5000	[ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=806370192 TSecr=2648514794
80	256.478575127	10.0.0.1	10.0.0.5	TCP	82	5000	→	43504	[PSH, ACK] Seq=1 Ack=1 Win=43520 Len=10 TSval=2648514795 TSecr=806370192
81	256.478604072	10.0.0.5	10.0.0.1	TCP	66	43504	→	5000	[ACK] Seq=1 Ack=17 Win=42496 Len=0 TSval=806370194 TSecr=2648514795
82	256.478661644	10.0.0.5	10.0.0.1	TCP	73	43504	→	5000	[PSH, ACK] Seq=1 Ack=17 Win=42496 Len=7 TSval=806370194 TSecr=2648514795
83	256.478665932	10.0.0.1	10.0.0.5	TCP	66	5000	→	43504	[ACK] Seq=17 Ack=8 Win=43520 Len=0 TSval=2648514796 TSecr=806370194
84	256.478970412	10.0.0.1	10.0.0.5	TCP	95	5000	→	43504	[PSH, ACK] Seq=17 Ack=8 Win=43520 Len=29 TSval=2648514796 TSecr=806370194
85	256.479232633	10.0.0.1	10.0.0.2	TCP	98	5000	→	32778	[PSH, ACK] Seq=102 Ack=6 Win=85 Len=32 TSval=2253722150 TSecr=2395549350
86	256.479302333	10.0.0.1	10.0.0.3	TCP	98	5000	→	54044	[PSH, ACK] Seq=117 Ack=29 Win=85 Len=32 TSval=4292691882 TSecr=778572233
87	256.479407014	10.0.0.1	10.0.0.4	TCP	98	5000	→	32778	[PSH, ACK] Seq=84 Ack=1 Win=85 Len=32 TSval=2707842885 TSecr=2935549689
88	256.479957157	10.0.0.2	10.0.0.1	TCP	66	35860	→	5000	[ACK] Seq=6 Ack=134 Win=83 Len=0 TSval=739996014 TSecr=285722150
89	256.479978852	10.0.0.1	10.0.0.2	TCP	99	5000	→	35860	[PSH, ACK] Seq=134 Ack=6 Win=85 Len=33 TSval=285722151 TSecr=739996014
90	256.479999145	10.0.0.2	10.0.0.1	TCP	66	35860	→	5000	[ACK] Seq=6 Ack=107 Win=83 Len=0 TSval=739996014 TSecr=285722151
91	256.480032969	10.0.0.3	10.0.0.1	TCP	66	54044	→	5000	[ACK] Seq=29 Ack=149 Win=83 Len=0 TSval=778575494 TSecr=4292691882
92	256.480038734	10.0.0.1	10.0.0.3	TCP	99	5000	→	54044	[PSH, ACK] Seq=149 Ack=29 Win=85 Len=33 TSval=4292691883 TSecr=778575494
93	256.480045730	10.0.0.3	10.0.0.1	TCP	66	54044	→	5000	[ACK] Seq=29 Ack=102 Win=83 Len=0 TSval=778575494 TSecr=4292691883
94	256.480051461	10.0.0.4	10.0.0.1	TCP	66	32778	→	5000	[ACK] Seq=1 Ack=116 Win=83 Len=0 TSval=2935549350 TSecr=2707842885
95	256.480056099	10.0.0.1	10.0.0.4	TCP	99	5000	→	32778	[PSH, ACK] Seq=116 Ack=1 Win=85 Len=33 TSval=2707842886 TSecr=2935549350
96	256.480061693	10.0.0.4	10.0.0.1	TCP	66	32778	→	5000	[ACK] Seq=1 Ack=149 Win=83 Len=0 TSval=2935549350 TSecr=2707842886
97	256.519947725	10.0.0.5	10.0.0.1	TCP	66	43504	→	5000	[ACK] Seq=8 Ack=46 Win=42496 Len=0 TSval=806370235 TSecr=2648514796
98	256.519975484	10.0.0.1	10.0.0.5	TCP	175	5000	→	43504	[PSH, ACK] Seq=46 Ack=8 Win=43520 Len=109 TSval=2648514837 TSecr=806370235
99	256.520033827	10.0.0.5	10.0.0.1	TCP	66	43504	→	5000	[ACK] Seq=8 Ack=155 Win=42496 Len=0 TSval=806370235 TSecr=2648514837

Figure 11.4: TCP segments corresponding to a private message exchange, showing point-to-point PSH, ACK delivery rather than broadcast fan-out.

11.4 Client Disconnection (FIN/ACK)

Figure 11.5 shows the connection teardown sequence for client 10.0.0.5: a FIN, ACK segment (frame 124) is sent from the client to the server, the server continues delivering any final PSH, ACK segments to remaining clients, and the exchange concludes with the server's FIN, ACK (frame 137) and the client's final ACK (frame 138), completing the graceful TCP connection close that corresponds to the client's /quit command and subsequent disconnection.

124	384.541789798	10.0.0.5	10.0.0.1	TCP	66	43504	→	5000	[FIN, ACK] Seq=13 Ack=155 Win=42496 Len=0 TSval=806498257 TSecr=2648514837
125	384.542781385	10.0.0.1	10.0.0.2	TCP	96	5000	→	35860	[PSH, ACK] Seq=181 Ack=6 Win=85 Len=30 TSval=285856214 TSecr=739224034
126	384.542822339	10.0.0.2	10.0.0.1	TCP	66	35860	→	5000	[ACK] Seq=6 Ack=211 Win=83 Len=0 TSval=739224077 TSecr=285856214
127	384.550143855	10.0.0.1	10.0.0.3	TCP	96	5000	→	54044	[PSH, ACK] Seq=196 Ack=29 Win=85 Len=30 TSval=4292819873 TSecr=778703514
128	384.550173974	10.0.0.3	10.0.0.1	TCP	66	54044	→	5000	[ACK] Seq=29 Ack=226 Win=83 Len=0 TSval=778703564 TSecr=4292819873
129	384.550189066	10.0.0.1	10.0.0.4	TCP	96	5000	→	32778	[PSH, ACK] Seq=163 Ack=1 Win=85 Len=30 TSval=2707970956 TSecr=2935677370
130	384.550542769	10.0.0.1	10.0.0.1	TCP	66	32778	→	5000	[ACK] Seq=1 Ack=193 Win=83 Len=0 TSval=2935677428 TSecr=2707970956
131	384.550585615	10.0.0.1	10.0.0.2	TCP	91	5000	→	35860	[PSH, ACK] Seq=211 Ack=6 Win=85 Len=25 TSval=285856222 TSecr=739224077
132	384.550594551	10.0.0.2	10.0.0.1	TCP	66	35860	→	5000	[ACK] Seq=6 Ack=230 Win=83 Len=0 TSval=739224085 TSecr=285856222
133	384.550599577	10.0.0.1	10.0.0.3	TCP	91	5000	→	54044	[PSH, ACK] Seq=226 Ack=29 Win=85 Len=25 TSval=4292819874 TSecr=778703564
134	384.550605745	10.0.0.3	10.0.0.1	TCP	66	54044	→	5000	[ACK] Seq=29 Ack=251 Win=83 Len=0 TSval=778703565 TSecr=4292819874
135	384.550609919	10.0.0.1	10.0.0.4	TCP	91	5000	→	32778	[PSH, ACK] Seq=193 Ack=1 Win=85 Len=25 TSval=2707970957 TSecr=2935677428
136	384.550612540	10.0.0.1	10.0.0.1	TCP	66	32778	→	5000	[ACK] Seq=1 Ack=219 Win=83 Len=0 TSval=2935677421 TSecr=2707970957
137	384.550746482	10.0.0.1	10.0.0.5	TCP	66	5000	→	43504	[FIN, ACK] Seq=155 Ack=14 Win=43520 Len=0 TSval=2648842668 TSecr=806498257
138	384.550770721	10.0.0.5	10.0.0.1	TCP	66	43504	→	5000	[ACK] Seq=14 Ack=156 Win=42496 Len=0 TSval=806498266 TSecr=2648842668

Figure 11.5: FIN, ACK segments exchanged between client 10.0.0.5 and the server, marking graceful TCP connection teardown after disconnection.

11.5 TCP Communication Over Port 5000

Across all captured frames, every segment exchanged between the clients and the server consistently uses source or destination port 5000 on the server side, confirming that all chat traffic, including handshakes, broadcasts, private messages, and disconnections, is carried over the single well-known port on which server.py listens.

11.6 Observed TCP Flags

- SYN: Sent by a client to initiate a new TCP connection to the server.
- SYN-ACK: Sent by the server in response to a SYN, acknowledging the client's request while simultaneously proposing its own sequence number.
- ACK: Sent by either side to acknowledge receipt of data or to complete the three-way handshake.

- PSH, ACK: Used whenever actual application data (login message, chat text, private message, or the ONLINE: user list) is being pushed to the receiving application immediately rather than buffered.
- FIN: Sent by a host to indicate it has finished sending data and wishes to close its side of the connection.
- FIN-ACK: Sent by the receiving host to acknowledge a FIN, and, in this capture, is emitted by the server when it closes its side of the connection after the client disconnects.

12. Reflection Questions

Why should networking logic and GUI code be separated?: Separating networking logic from GUI code keeps each concern independently testable and maintainable: the socket-handling code can be reused, as demonstrated by carrying `server.py` forward unchanged from Assignment 5, while the presentation layer can be redesigned or replaced (for example, swapping Tkinter for another toolkit) without touching the protocol implementation. It also prevents blocking network calls from directly coupling to, and potentially freezing, the user interface thread.

Why is a background thread required?: A background thread is required because Tkinter's `mainloop()` and the blocking socket call `client.recv()` cannot both run on the same thread without one starving the other. Without a separate thread, the GUI would become unresponsive every time the client waited for data from the server, since `recv()` only returns once a message arrives.

Which components from Assignment 5 were reused?: The TCP server (`server.py`) was reused in its entirety, including the client dictionary and thread-per-client model, the `broadcast()` and `private_message()` functions, the `save_history()` chat logging mechanism, the `send_online_users()/update_online_users()` online-user broadcasting logic, and the overall threading and locking strategy used to protect shared state. Only the client-side presentation layer was newly built.

How did the GUI improve usability?: The GUI improved usability by replacing raw command syntax and mixed console output with clearly labelled fields (Server IP, Username), dedicated visual regions for chat history and online users, one-click buttons for Connect, Send,

and Disconnect, an Enter-to-send keybinding, and a color-coded connection status indicator, all of which make the application approachable without requiring the user to memorize commands or read interleaved log output.

Suggest one future enhancement.: A useful future enhancement would be to let the user select a private-message recipient directly from the Online Users Listbox (for example, by double-clicking a username to pre-fill the message box with `/msg <username>`), removing the need to manually type the recipient's name and reducing the chance of typos when addressing private messages.

13. Conclusion

Assignment 6 successfully converted the terminal-based TCP chat client from Assignment 5 into a fully functional GUI-based desktop application using Python Tkinter, while reusing the existing TCP server implementation without modification to its core networking logic. The resulting `client_gui.py` application provides a login window, a scrollable chat display, an automatically updating online user list, broadcast and private messaging, and a responsive interface backed by a dedicated background receiving thread. Testing within a Mininet single,5 topology using four simultaneous clients confirmed correct behaviour for login, broadcast messaging, private messaging, user join/leave notifications, disconnect handling, and background message reception, and Wireshark packet captures on `tcp.port == 5000` verified correct TCP connection establishment, message delivery, and connection teardown at the protocol level. All assignment objectives were successfully achieved.

14. References

- Python Software Foundation, “Python 3 Documentation,” <https://docs.python.org/3/>
- Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” <https://docs.python.org/3/library/tkinter.html>
- Mininet Project, “Mininet Documentation,” <http://mininet.org/>
- Wireshark Foundation, “Wireshark User's Guide,” <https://www.wireshark.org/docs/>